game.py
Code

README.md
Document

Analyze this game.py file briefly. Explain the main game logic and identify the deliberate height-score bug and the 3 incomplete functions. Don't change any code yet.

1. Main game logic

`game.py` is a vertical climbing game built using **Pygame**, where the player jumps between platforms, collects coins, and tries to reach the top.

- **Player movement:** Arrow keys or A/D move left and right. Space, W, or Up makes the player jump. Gravity pulls the player down.
- **Platform generation:** Platforms are randomly placed at different heights. The ground platform stays stationary.
- **Collision detection:** The player lands on a platform only when falling onto it from above.
- **Camera:** The camera scrolls upward as the player climbs and never scrolls back down.
- **Coins and scoring:** Coins spawn randomly on platforms. Collecting one adds 50 points.
- **Lives and respawning:** Falling costs a life. The player respawns at their last safe position, and the game ends when no lives remain.
- **Win condition:** Reaching the topmost platform wins the game.

2. The deliberate height-score bug

Location: `Game.update()`, around lines 155–156.

Python

```python
current_height = max(0, (HEIGHT - 40 - self.player.rect.y) // 10)
self.height = current_height
```

Problem: `self.height` is overwritten every frame using the player's current position. If the player climbs higher and subsequently falls, their displayed height decreases.

Expected behavior: Height should record the *maximum height reached so far* during the current run and never decrease.

For example, if the player reaches 40 m and falls back to 25 m, the displayed height should remain 40 m.

3. The three incomplete functions

All three functions currently contain only `pass`, meaning their implementations are missing.

1. `platform_color(index, total)`

Purpose: Assign colors to platforms based on their position in the level.

- Receives the platform's index and total platform count.
- Should return an RGB color tuple or `None` for the default green.
- Suggested feature: gradually change colors as the player climbs higher.

---

2. `moving_platform_speed(index, total)`

Purpose: Decide which platforms move horizontally.

- Returns a movement speed or `None` / `0` for stationary platforms.
- The existing `Platform.update()` already handles movement and bouncing.
- Suggested feature: make every third platform move.

3. `on_coin_collected(coin, score)`

Purpose: Add a response when the player collects a coin.

- Receives the collected coin and the updated score.
- Called immediately after the coin's 50 points are added.
- Suggested feature: add a sparkle effect or a coin-collection sound.

4. Quick summary

Item	Current state	Required change
Height score	Buggy	Preserve the maximum height reached

Platform colors	Incomplete	Return colors based on platform index
Moving platforms	Incomplete	Select platforms and assign movement speeds
Coin collection hook	Incomplete	Add a collection effect or bonus behavior

Overall: The core game mechanics are already implemented. The lab requires one scoring fix and three small feature implementations. No code has been changed, as requested.

Fix Task 1: the height score should track the maximum height reached and never decrease. Show me the smallest code change, explain it briefly, and tell me how to test it.

1. Smallest code change

In `Game.update()`, find these lines around line 155:

Python

```
current_height = max(0, (HEIGHT - 40 - self.player.rect.y) // 10)
self.height = current_height
```

Replace them with:

Python

```
current_height = max(0, (HEIGHT - 40 - self.player.rect.y) // 10)
self.height = max(self.height, current_height)
```

2. Why this works

`max(self.height, current_height)` keeps whichever value is larger: the previously achieved height or the current height.

For example:

Current height	Displayed height
10 m	10 m
25 m	25 m
40 m	40 m
20 m	40 m
30 m	40 m

The score increases when the player reaches a new height but never decreases during that run. The existing `reset()` method sets `self.height = 0`, so restarting begins a fresh run.

3. How to test it

1. Run the game using `python game.py`.
2. Jump across platforms and observe the **Height** value in the HUD.
3. Climb to a higher platform and note the displayed height.
4. Fall to a lower platform or backtrack.
5. Verify that the height stays at its previous maximum.
6. Press `R` to restart and verify that the height resets.

Expected result: Height increases when a new maximum is reached, remains unchanged when falling, and resets to zero on restart.

Only these two lines need changing. The three incomplete functions can remain untouched for now.

For task 2: Implement platform_color(index, total) so platform colors gradually change as we climb higher. Keep it simple, handle edge cases, and explain the code briefly.

Replace the `pass` inside `platform_color(index, total)` with this simple implementation:

Python

```
def platform_color(index, total):
    """Return a color that gradually changes with platform height."""
    if total <= 0:
        return None
```

```
    progress = max(0, min(1, index / total))

    red = int(100 + 155 * progress)
    green = int(180 - 100 * progress)
    blue = int(100 + 155 * progress)

    return (red, green, blue)
```

How it works

- `progress` represents how high the platform is in the level, from `0` to `1`. The `max` and `min` keep it within that range.
- **Red and blue** gradually increase from 100 to 255.
- **Green** gradually decreases from 180 to 80.
- **Edge case:** If `total <= 0`, the function returns `None`, so the game uses its default green color.

The existing `Platform.__init__()` already calls this function and applies the returned RGB color, so you don't need to change anything else.

How to test

1. Run `python game.py`.
2. Look at the platforms as you climb.
3. Verify that lower platforms are greener and higher platforms gradually become more purple/pink.

That's it — one function, no extra libraries, and no changes to the existing game logic.

Implement moving_platform_speed(index, total) so every third eligible platform moves slowly while the ground stays static. The movement logic already exists. Show the function and explain how to test it.

Replace the `pass` inside `moving_platform_speed(index, total)` with this:

Python

```
def moving_platform_speed(index, total):
    """Make every third eligible platform move slowly."""
```

```
    if total <= 0 or index <= 0:
        return None

    if index % 3 == 0:
        return 1

    return None
```

How it works

- `index <= 0`: Keeps the ground platform static.
- `index % 3 == 0`: Makes every third platform (3, 6, 9, ...) move.
- `return 1`: Moves the platform at **1 pixel per frame**.
- `return None`: Leaves other platforms stationary.

The existing `Platform.update()` handles horizontal movement and bouncing between boundaries. The ground is also explicitly created with `movable=False`, so it stays static regardless.

How to test

1. Run `python game.py`.
2. Observe the platforms while climbing.
3. Look for every third platform moving slowly from side to side.
4. Verify that other platforms remain stationary and the ground does not move.
5. Jump onto a moving platform and check that the player is carried along as it moves.

Expected result: Some platforms move gently, the others remain still, and the existing collision and movement logic continues to work without additional changes.

For final task: Implement on_coin_collected(coin, score) with a simple visible effect or coin combo counter. Inspect the existing code first to ensure it actually work

I inspected the existing `game.py` logic. The simplest reliable choice is a **visible sparkle effect** when a coin is collected.

One important detail: `on_coin_collected(coin, score)` is a standalone function, not a method of `Game`. It receives the coin's position and the updated score, but it does not receive the Pygame screen or have access to the main drawing loop. So drawing directly inside the function won't work reliably—the effect needs to be stored and drawn during the game's normal rendering.

1. Add a sparkle effect

Step 1: Add this global variable near the constants at the top of `game.py`:

Python

```
coin_sparkles = []
```

Step 2: Replace `on_coin_collected()` with:

Python

```
def on_coin_collected(coin, score):
    """Create a short sparkle effect at the coin's position."""
    coin_sparkles.append(
        (coin.pos.copy(), pygame.time.get_ticks())
    )
```

This stores the coin's position and the time it was collected. The `score` parameter is available if you want to add more behavior later.

2. Draw the effect

Inside `Game.draw()`, add this code **after drawing the player and before rendering the HUD**:

Python

```
        now = pygame.time.get_ticks()
        active_sparkles = []

        for pos, start_time in coin_sparkles:
            age = now - start_time

            if age < 400:
                center = (int(pos.x), int(pos.y - self.cam_y))
```

```
                radius = 7 + age // 30
                pygame.draw.circle(
                    screen, (255, 255, 150), center,
                    radius, 2
                )
                active_sparkles.append((pos, start_time))

        coin_sparkles[:] = active_sparkles
```

3. Why this works

- **Collection hook:** Your existing `Game.update()` already calls `on_coin_collected(coin, self.score())` immediately after collecting a coin.
- **Visible effect:** `Game.draw()` draws an expanding yellow ring at the coin's former position.
- **Camera support:** Subtracting `self.cam_y` keeps the effect aligned with the scrolling game world.
- **Automatic cleanup:** Each sparkle disappears after 400 milliseconds, and expired effects are removed from the list.

4. How to test it

1. Run `python game.py`.
2. Jump into a coin.
3. Observe a short, expanding yellow ring where the coin was.
4. Collect multiple coins and verify that each produces its own effect.
5. Continue climbing to check that the effect stays aligned while the camera scrolls.

One small note: This effect requires the two additions above—the function alone cannot draw on the screen because it doesn't receive the `screen` object. No other game mechanics need to change.

Briefly check all 4 tasks and tell me what I should test manually before submitting.. Are all tasks done?

Based on the original `game.py` and the changes we've discussed, **all 4 tasks have proposed implementations**, but Task 4 needs one extra check: its sparkle effect requires both the collection hook and the drawing code. I can't confirm they're all present in your actual file unless I inspect your latest saved version.

Manual testing checklist

Pre-submission checklist 0/12 checked

Task 1 — Height score

- [ ] Climb higher and confirm height increases.
- [ ] Fall or backtrack and confirm height never decreases.
- [ ] Press R and confirm height resets for the new run.

Task 2 — Platform colors

- [ ] Run the game and compare lower and higher platform colors.
- [ ] Confirm colors change gradually, without invalid RGB values.

Task 3 — Moving platforms

- [ ] Confirm every third platform moves slowly.
- [ ] Confirm other platforms and the ground stay static.
- [ ] Stand on a moving platform and check that the player is carried along.

Task 4 — Coin sparkle

- [ ] Collect a coin and confirm the yellow ring appears.
- [ ] Collect multiple coins and confirm each triggers an effect.
- [ ] Wait for the rings to disappear and check alignment while the camera scrolls.
- [ ] Confirm each coin still adds 50 points and disappears.

Are all tasks done?

Not in the actual uploaded file yet. I checked it, and it still has the original height-score bug and all three `pass` functions. The changes we've discussed need to be applied to your local `game.py` and saved.

Before submitting, also verify these three things:

- Task 1: change the height assignment to `self.height = max(self.height, current_height)`.
- Tasks 2–3: replace both `pass` statements with the color and movement implementations.
- Task 4: add the `coin_sparkles` list, implement `on_coin_collected()`, and add the drawing/cleanup code inside `Game.draw()`.

Finally, run the game from start to finish at least once. Check that you can still win, lose all lives, restart, collect coins, and climb without crashes.

Your README also asks for a **10-second before video, a 10-second after video, and the complete chat history link**. Don't forget those submission requirements.